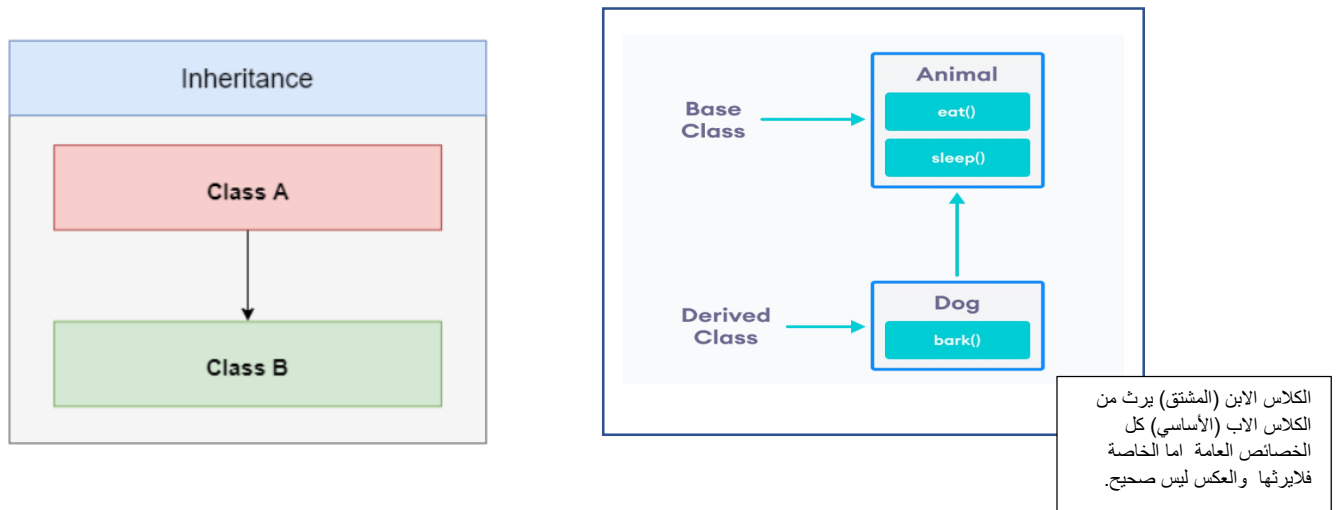


4-1 INHERITANCE

- Inheritance is probably the most powerful feature of object-oriented programming, after classes themselves. Inheritance is the process of creating new classes, called derived classes, from existing or *base classes*.
- The mechanism of deriving a new class from an old one is called **inheritance (or derivation)**.
- The derived class inherits all the capabilities of the base class but can add refinements of its own. The base class is unchanged by this process. The inheritance relationship is shown in Figures bellow.



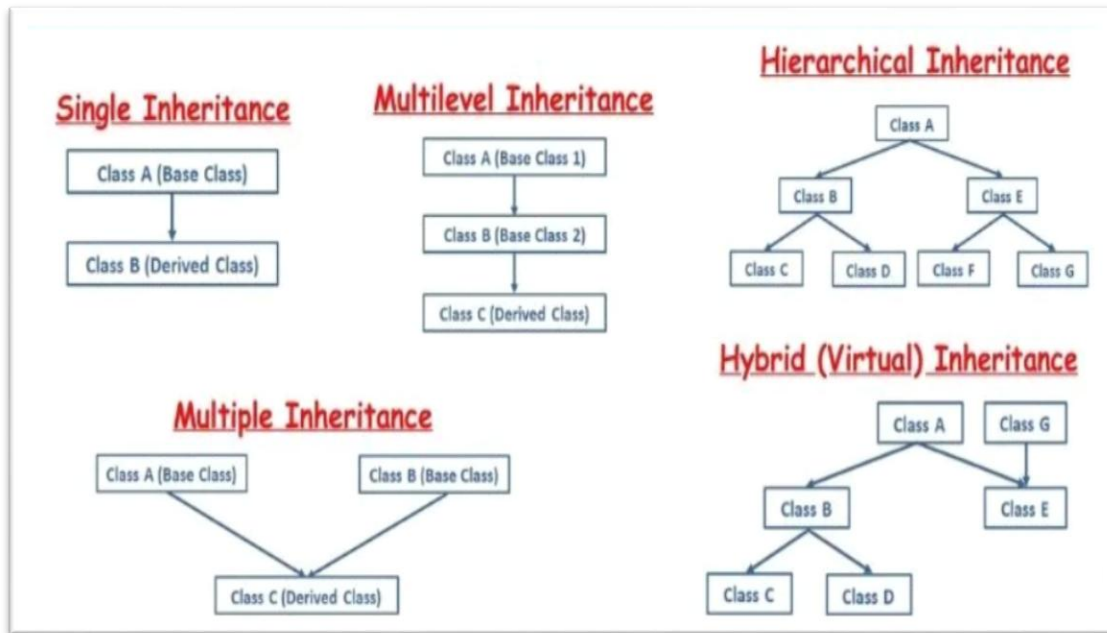
- Inheritance is an essential part of OOP. Reusing existing code saves time and effort and increases a program's reliability.
- A programmer can use a class created by another person or company, and, without modifying it, derive other classes from it that are suited to particular situations.

4-2 Inheritance Forms.

There are various forms of inheritance:

- Single inheritance:** a derived class with only one base class.
- Multiple inheritance:** one derived class with several base classes.
- Hierarchical inheritance:** one base class may be inherited by more than one derived class.

- 4- **Multilevel inheritance:** The mechanism of deriving a class from another “derived class”.
- 5- **Hybrid inheritance:** is a combination of two or more types of inheritance (such as single, multiple, multilevel, or hierarchical) within a single class hierarchy.



4-3 Defining Derived Classes

A derived class is defined by specifying its relationship with the base class in addition to its own details. The general syntax of defining a derived class is:

```
class derived-class-name : visibility-mode base-class-name
{
    . . . // members of derived class
};
```

The colon indicates that the ***derived-class-name*** is derived from the ***base-class-name***. The visibility mode is optional and, if present, may be either private or public. The default ***visibility-mode*** is private. Visibility mode specifies whether the features of the base class are ***private derived*** or ***public derived***.

Example:-

```
class child : private parent           // private derivation
{
```

```

members of parent
};
class child : public parent      // public derivation
{
members of parent
};

```

4-4 C++ Inheritance Access

- Inheritance access defines how the access specifiers (public, protected, private) of a base class are treated in the derived class.
 - It controls the visibility and accessibility of base class members in the derived class and to outside code.
 - The type of inheritance (public, protected, private) determines how the base class members are inherited.

Types of Inheritance Access

- **Public:** Base class members retain their access levels in the derived class (public → public, protected → protected).
- **Protected:** Base class members become protected in the derived class, regardless of original access (public/protected → protected).
- **Private:** Base class members become private in the derived class, restricting access outside the derived class.

Accessibility	Public Members	Protected Members	Private Memembers
Base Class	Yes	Yes	Yes
Derived Class	Yes	Yes	No

Example about public Inheritance

In this example, public inheritance is demonstrated. Since private and protected members will not be directly accessed from main() so we have had to create functions name getPVT() to access the private variable and getProt() to access the protected variable from the inherited class.

```
#include <iostream>
using namespace std;

class Base
{
    private:
        int pvt = 1;

    protected: ←
        int prot = 2;

    public:
        int pub = 3;

        // function to access private member
        int getPVT()
        {
            return pvt;
        }
};

class PublicDerived : public Base
{
    public:
        // function to access protected member from Base
        int getProt()
        {
            return prot;
        }
};

int main()
{
    PublicDerived object1;
    cout << "Private = " << object1.getPVT() << endl;
    cout << "Protected = " << object1.getProt() << endl;
    cout << "Public = " << object1.pub << endl;
    return 0;
}
```

المتغير المعرف على انه protected لا يمكن الوصول اليه في الدالة الرئيسية فقط في موضوع الوراثة يمكن الوصول اليه من خلال الكلاس المشتق .

Output:
Private = 1
Protected = 2
Public = 3

Example about Protected Inheritance

We know that protected members can only be accessed from the Derived class. These members cannot be directly accessed from outside the class. So we cannot use getPVT() from ProtectedDerived. This is also why we need to create getPub() function in the Derived class in order to access the pub variable.

```
#include <iostream>
using namespace std;

class Base
{
private:
    int pvt = 1;

protected:
    int prot = 2;

public:
    int pub = 3;

    // function to access private member
    int getPVT()
    {
        return pvt;
    }
};

class ProtectedDerived : protected Base
{
public:
    // function to access protected member from Base
    int getProt()
    {
        return prot;
    }

    // function to access public member from Base
    int getPub()
    {
        return pub;
    }

    // function to get access to private members from Base
    int try_getPVT()
    {
        return Base::getPVT();
    }
};
```

Output:

Private = 1

Protected = 2

Public = 3

```
};  
  
int main()  
{  
    ProtectedDerived object1;  
    cout << "Private = " << object1.try_getPVT() << endl;  
    cout << "Protected = " << object1.getProt() << endl;  
    cout << "Public = " << object1.getPub() << endl;  
    return 0;  
}
```

Note: There is still a way to access the Private members of base class. Since Base has a public member function that can access its private member variables, we can create a call to this member function (inherited as Protected member function in derived class) to access the Private inherited variable of Base class.

Example about Private Inheritance

We know that private members cannot be accessed from the Derived class. These members cannot be directly accessed from outside the class. So we cannot use getPVT() from PrivateDerived. So we need to create the getPub() function in PrivateDerived in order to access the pub variable.

```
// C++ program to demonstrate the working of private inheritance  
  
#include <iostream>  
using namespace std;  
  
class Base {  
    private:  
        int pvt = 1;  
  
    protected:  
        int prot = 2;  
  
    public:  
        int pub = 3;  
  
    // function to access private member  
    int getPVT() {  
        return pvt;  
    }  
};
```

```
class PrivateDerived : private Base {
public:
    // function to access protected member from Base
    int getProt() {
        return prot;
    }

    // function to access private member
    int getPub() {
        return pub;
    }
};

int main() {
    PrivateDerived object1;
    cout << "Private cannot be accessed." << endl;
    cout << "Protected = " << object1.getProt() << endl;
    cout << "Public = " << object1.getPub() << endl;
    return 0;
}
```

Where are the Real-World Applications of Inheritance?

- **Medical Systems**
Person → Patient, Doctor, Nurse
- **Banking Systems**
Account → SavingAccount, CurrentAccount
- **Vehicle Systems**
Vehicle → Car, Truck, Motorcycle
- **Game Development**
Character → Player, Enemy, Boss
- **University Systems**
Person → Student, Teacher
- **GUI Applications**
Component → Button, TextBox, CheckBox